

CSCI-121 · Week 2, Day 2

Name _____ Section (circle): R3 Mon / R4 Wed

The rule from today

Casting truncates. Formatting rounds. Choose on purpose.

```
double v = 3.999;

(int) v           // 3
String.format("%.0f", v) // "4"
```

Strings — the set worth memorising

Java	Python
<code>s.length()</code>	<code>len(s)</code>
<code>s.charAt(i)</code>	<code>s[i]</code>
<code>s.substring(a, b)</code>	<code>s[a:b]</code> — stops before <code>b</code>
<code>s.equals(t)</code>	<code>s == t</code>
<code>s.toUpperCase()</code>	<code>s.upper()</code>
<code>s.trim()</code>	<code>s.strip()</code>
<code>s.contains(t)</code>	<code>t in s</code>

No negative indexing. The last character is `s.charAt(s.length() - 1)`.

Every one of these returns a new String. None of them change the one you had.

Predict, then we run it

	Code	Your prediction
1	<code>String s = "hi"; s.toUpperCase(); print(s);</code>	
2	<code>print("hello".substring(0, 3));</code>	
3	<code>print("hello".charAt(4));</code>	
4	<code>print((int) 2.99);</code>	
5	<code>print(String.format("%.0f", 2.99));</code>	
6	<code>print(((int)(1.4815 * 100)) / 100.0);</code>	
7	<code>print(((int)(1.4815 * 100)) / 100);</code>	

6 and 7 differ by one character. Say why:

The two-decimal trick — SB1 needs this

```
double v = 1.4815297665908702;  
double t = ((int)(v * 100)) / 100.0;
```

Fill in each step:

Step	Expression	Value
1	<code>v * 100</code>	
2	<code>(int)(v * 100)</code>	
3	<code>... / 100.0</code>	

printf VS String.format

```
System.out.printf("%.2f%n", v); // _____  
String out = String.format("%.2f", v); // _____
```

Which one would you use inside a method that has to **return** the text?

Compiling — what the terminal showed us

```
javac Hello.java
```

What new file appeared? _____

Is it for you to read? _____

The first four bytes of every Java class file ever compiled: _____

```
java Hello
```

Why is there no `.java` or `.class` on that command?

Compile time vs run time

	When it happens	Example
Compile time		
Run time		

TimeUtils — with your neighbour

3725 seconds. Fill in the blanks so this returns `01:02:05`.

```

public static String formatTime(int totalSeconds) {

    int hours    = totalSeconds / 3600;

    int rem      = totalSeconds % 3600;

    int minutes  = rem / 60;

    int seconds  = rem % 60;

    return String.format("%02d:%02d:%02d", hours, minutes, seconds);
}

```

3725 / 3600 is 1, not 1.03. Here integer division is not something you are working around — it is the tool. Why?

Exit ticket — hand this in at the door

1. `(int) 2.99` and `String.format("%.0f", 2.99)` — give both answers.
 2. You run `javac` and get an error. How many `.class` files were produced?
 3. One thing from Tuesday that makes more sense now than it did on Tuesday.
-
-

This week and next

Skill Builder 1	Sun Sep 13, 11:59 PM — today was its content
Drills — Asterisks · Show me the Numbers	Sun Sep 6, 11:59 PM
Recitation 2 — finish at home, push, open the PR	Sun Sep 6, 11:59 PM
Reading — Think Java Ch 2, Ch 3	
§6.10 <i>String Comparison</i> · §9.3 <i>Strings Are Immutable</i>	the two sections that cover today and Tuesday

R3 — no section on Mon Sep 7. Labor Day. Your next section is **Sep 14**, and SB1 is due Sep 13. Use office hours and tutoring, and re-read **Recitation 2 exercises 6, 7 and 8** — they are SB1's spine.